

MONOPOLY: Product Line Analysis via Monotonicity

Logan Murphy¹, Marc Carwehl², Juliane Päßler³, and Marsha Chechik¹

¹ University of Toronto, Toronto, Canada
{lmurphy, chechik}@cs.toronto.edu

² Humboldt Universität zu Berlin, Berlin, Germany
m.carwehl@hu-berlin.de

³ TU Wien, Vienna, Austria
juliane.paessler@tuwien.ac.at

Abstract. Software product lines (SPLs) model families of related products. Analysis techniques for single products have been *lifted* to SPLs. However, lifted analyses are often specific to the domain of the SPL, and the type of analysis. In this work, we introduce MONOPOLY, a general framework for black-box lifting of a wide class of analyses for different SPL domains. We provide a detailed formalization of SPL domains and their analyses that can be lifted, including a correctness theorem, and a proof thereof. Through an empirical analysis, we showcase the potential of our framework on two SPLs from the literature with several thousand products and report on a complete analysis of the SPL by analyzing only between 3.6% and 24% of all SPL products.

Keywords: Software Product Lines · Lifted Analysis · Formal Methods.

1 Introduction

To address the increasingly significant problem of managing software-based systems with inherent variability, the field of software product line engineering (SPLE) [22] has developed a wealth of techniques for defining, modeling, and analyzing related software products, organized into *software product lines* (SPLs).

A classic problem in SPLE is the *analysis* of the products in an SPL [23]. Since SPLs can encompass very many distinct products, analyzing each product individually via traditional methods is generally infeasible. One option is of course to analyze only a small number of products, but a lack of completeness of the analysis over the entire product space may be unacceptable in some circumstances, e.g., for safety-critical SPLs. Another option, which has been studied extensively by the SPLE community, is to develop specialized *lifted* analyses in a white-box fashion. These analyses are able to handle variability semantics directly during evaluation, thus making the analysis of the entire product space tractable. While lifted analyses can offer significant improvements in efficiency, their implementation and evaluation for correctness can be a significant engineering challenge, as it requires in-depth knowledge of both the product-based

analysis being lifted and of SPL semantics [19]. This is even further complicated if the product-based analysis being lifted is closed-source or proprietary. Thus, to encourage the adoption of SPLE techniques, there is a need for analysis strategies which:

- (i) are more efficient than individual product-based analysis,
- (ii) do not sacrifice completeness of analysis over the product space, and
- (iii) allow engineers to reuse existing (and trusted) product-based analyses.

Following [19], we refer to analyses satisfying these criteria as *black-box lifts*. While various such lifted analyses have been developed [3, 15, 12], a limitation of existing techniques is that they are typically *ad hoc*, in the sense that the lifting is done with respect to a specific analysis for a specific type of SPL. In modern SPL engineering, there are many *SPL domains*, which need to undergo many kinds of analysis: SPLs of code [14], but also of models [7], specifications [8], languages [11], proofs [6], assurance cases [20], and so on. As such, we argue that the ability to lift entire *classes* of analyses in a black-box fashion could significantly facilitate the adoption of SPLE techniques.

To this end, we introduce the MONOPOLY⁴ framework for black-box lifting of a wide class of analyses for different SPL domains. MONOPOLY is based on the observation (cf. Sec. 4) that an analysis of one specific product often generalizes to other products. Thus, MONOPOLY uses the analysis result of a product to produce a set of products for which this analysis result can be generalized. We formalize our observation using elementary concepts from order theory, defining a general procedure for lifting product-based analyses in a black-box fashion. Additionally, we provide a tool-based framework for black-box lifting. To instantiate our framework, users must only implement two well-defined operators for the desired SPL domain.

Contributions. Our concrete contributions in this work are as follows:

1. We introduce the framework MONOPOLY which iteratively generalizes the analysis result of a single product to a set of products, thereby providing a black-box approach for lifting product-based analysis to SPLs.
2. We provide a detailed formalization of MONOPOLY and prove a general correctness theorem which applies to all instantiations of the framework.
3. We showcase MONOPOLY for three different SPL domains: sets [9], assume-guarantee contracts [21], and metamodels [13].
4. We provide an implementation of MONOPOLY which allows users to instantiate the framework for analyses over an SPL of their choice.
5. We evaluate MONOPOLY’s efficiency relative to individual analysis of every product using existing benchmark SPLs from the literature [13].

⁴ (Mono)tonic (p)r(o)duct (l)ine anal(y)sis.

Organization. The remainder of this paper is structured as follows: we begin by establishing some preliminary concepts from SPL analysis and order theory (Sec. 2). Then, we define the MONOPOLY framework, prove its correctness, and describe our Python implementation and how it can be instantiated (Sec. 3). Next, we present the methodology and results from our empirical evaluation of MONOPOLY (Sec. 4), including a discussion of its strengths and limitations, followed by a consideration of related work (Sec. 5) before concluding in Sec. 6.

2 Background

In this section, we recall some basic concepts from SPL analysis and order theory which are necessary to define the MONOPOLY framework.

2.1 Annotative Software Product Lines

There are numerous paradigms for representing families of related artifacts as SPLs [19]. In this work, we operate within the *annotative* paradigm [26], in which elements of software artifacts (lines of code, model elements, etc.) are *annotated* to indicate in which product(s) they are used. Each *product* in the SPL is produced by specifying a *configuration* of features, which are atomic units of functionality that can be present or absent in any product. Formally, given a set of features F , a *configuration* is a subset $\mathbf{c} \subseteq F$ of features present in the product. Features are also used to define *feature expressions*, i.e., propositional logic formulae whose atomic propositions are features. For instance, given $F = \{A, B, C\}$, the formulae $\neg A$, $B \vee C$, and $A \Rightarrow \neg C$ are all feature expressions over F . We let $\text{Prop}(F)$ denote the set of feature expressions over F . By interpreting a configuration $\mathbf{c}$ as a truth valuation which maps all $f \in \mathbf{c}$ to $\top$ and all other features to $\perp$, we have a natural entailment relation $\mathbf{c} \models \phi$ between configurations $\mathbf{c}$ and feature expressions ϕ . Given any $\phi \in \text{Prop}(F)$, we let $\llbracket \phi \rrbracket$ denote the set of configurations of F which satisfy ϕ .

Not all possible combinations of features may correspond to valid products (e.g., some features may be mutually exclusive). Thus, defining an SPL requires specifying a *feature model* describing which configurations of features are considered valid. There are numerous ways of representing a feature model, e.g., feature diagrams [17] or via the Universal Variability Language [1]. In this work, we assume a feature model is given as a feature expression Φ , such that $\llbracket \Phi \rrbracket$ represents the set of configurations corresponding to valid products.

Annotative SPLs can be constructed for a wide variety of domains. In general, an annotative SPL is defined by constructing an object $D \in \mathbb{D}$, called the *domain model*, which includes *all* elements needed in any potential product, and then annotating each element of D with a feature expression – called a *presence condition* – defining in which configurations the element is present.

Definition 1 (Annotative SPL [26]). An *annotative SPL* over domain $\mathbb{D}$ is a tuple $\mathcal{L} = \langle F, \Phi, D, \pi \rangle$, where F is a set of features, Φ is a feature model over

$F, D \in \mathbb{D}$ is the domain model, and π is a function assigning to each element e of D a presence condition $\pi(e) \in \text{Prop}(F)$, defining the set of configurations in which e is present.

A *product* is an object which is *derived* from an SPL given a choice of a feature configuration: Given an SPL $\mathcal{L} = \langle F, \Phi, D, \pi \rangle$ over a domain $\mathbb{D}$, and a configuration $\mathfrak{c} \subseteq F$, the *product derived from $\mathcal{L}$ under $\mathfrak{c}$* (denoted $\mathcal{L}|_{\mathfrak{c}}$) is obtained by removing all elements e from the domain model D such that $\mathfrak{c} \not\models \pi(e)$. More generally, given a feature expression ϕ over F , the *partial derivation* of $\mathcal{L}$ under ϕ (denoted $\mathcal{L} \downarrow \phi$) is the set of products derivable from $\mathcal{L}$ by configurations satisfying ϕ . That is, $\mathcal{L} \downarrow \phi = \{\mathcal{L}|_{\mathfrak{c}} \mid \mathfrak{c} \models \phi\}$.

Example 1 (SPL of Sets). Fix the universe of natural numbers $\mathbb{N}$ and consider the SPL $\mathbf{S} = \langle F, \Phi, S, \pi \rangle$ over the domain $\mathcal{P}(\mathbb{N})$, i.e., the products of $\mathbf{S}$ are sets of natural numbers. Let $F = \{A, B, C\}$, $\Phi = \text{true}$ (no constraints among features), $S = \{1, 2, 3, 4\} \in \mathcal{P}(\mathbb{N})$, and $\pi(1) = A, \pi(2) = B, \pi(3) = C, \pi(4) = \neg(A \vee C)$. Given, for example, the configuration $\mathfrak{c} = \{B, C\} \subseteq F$, the derived product is $\mathbf{S}|_{\mathfrak{c}} = \{2, 3\} \in \mathcal{P}(\mathbb{N})$. Fig. 1 visualizes all derived products as nodes.

Our second example is based on *assume-guarantee* (AG) contracts [25]. AG contracts are used to support modular system design by specifying what behaviours can be expected from a system's environment (assumptions), and what behaviours can be provided by the system (guarantees) under those conditions. Assumptions and guarantees can be specified using a wide variety of languages, such as quantifier-free formulas over real arithmetic or temporal logics. We assume that a specification language for contracts has been fixed. Formally, a contract K is a pair $\langle \{A_1, \dots, A_n\}, \{G_1, \dots, G_m\} \rangle$ where the A_i are assumptions and the G_i are guarantees. We denote the domain of all possible AG contracts (expressed using the fixed specification language) as $\mathbb{K}$. SPLs over $\mathbb{K}$, i.e. SPLs with AG contracts as products, SPLs of AG contracts have previously been studied, e.g., [21, 18].

Example 2 (SPL of Assume-Guarantee Contracts). Consider the (slightly modified) example of an SPL of AG contracts $\mathbf{K}_{auv} = \langle F_{auv}, \Phi_{auv}, K_{auv}, \pi_{auv} \rangle$ for autonomous underwater vehicles (AUVs) from [18]. The SPL is defined over features $F_{auv} = \{vhf, sat, oxy, cam\}$ and with feature model $\Phi_{auv} = (vhf \vee sat) \wedge \neg(vhf \wedge sat) \wedge (oxy \vee cam) \wedge (cam \Rightarrow vhf)$, reflecting that an AUV can communicate either via VHF radio (*vhf*) or via satellite (*sat*), and that it can do environmental surveillance with an oxygen sensor (*oxy*) or a camera (*cam*). The domain model $K_{auv} = \langle \{A\}, \{G_1, G_2\} \rangle$ contains all possible assumptions and guarantees, with presence conditions $\pi_{auv}(A) = vhf$, $\pi_{auv}(G_1) = oxy$, and $\pi_{auv}(G_2) = cam \vee sat$. For example, a product derived from a configuration including *vhf* contains the assumption A . The feature model Φ_{auv} allows four different valid configurations: $c_1 = \{vhf, cam\}$, $c_2 = \{vhf, cam, oxy\}$, $c_3 = \{vhf, oxy\}$, and $c_4 = \{sat, oxy\}$. We denote by K_{c_i} the product $\mathbf{K}_{auv}|_{c_i}$ derived from c_i for $i \in \{1, \dots, 4\}$, e.g., $K_{c_1} = \{\{A\}, \{G_2\}\}$.

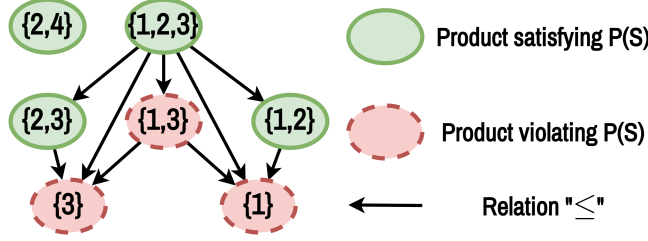


Fig. 1: Products derived from Example 1 and their relations with respect to “ $\leq$ ”.

This work is concerned with the *analysis* of SPLs. More specifically, we consider the lifting of Boolean analyses from products to SPLs. Given some fixed domain $\mathbb{D}$ and property $P : \mathbb{D} \rightarrow \{\top, \perp\}$, we assume the existence of an analysis A which, for any $X \in \mathbb{D}$, returns $\top$ if $P(X)$ holds and returns $\perp$ otherwise. We refer to such an analysis A as being a *P-analysis* of $\mathbb{D}$.

Definition 2 (Lifted SPL Analysis). Let $P : \mathbb{D} \rightarrow \{\top, \perp\}$ be a property of elements of domain $\mathbb{D}$, and let A be a P -analysis of $\mathbb{D}$. A *lift* of A , written $A^\uparrow$, is a P -analysis that can be applied to any SPL $\mathcal{L}$ over $\mathbb{D}$ such that the following properties hold: $A^\uparrow(\mathcal{L})$ returns a pair $\langle \Phi_\top, \Phi_\perp \rangle$ of feature expressions such that $\llbracket \Phi_\top \rrbracket \cap \llbracket \Phi_\perp \rrbracket = \emptyset$, $\llbracket \Phi_\top \rrbracket \cup \llbracket \Phi_\perp \rrbracket = \llbracket \Phi \rrbracket$, $(\mathcal{L} \downarrow \Phi_\top) \subseteq \{X \in \mathbb{D} \mid P(X)\}$, and $(\mathcal{L} \downarrow \Phi_\perp) \subseteq \{X \in \mathbb{D} \mid \neg P(X)\}$.

That is, a P -analysis of an SPL $\mathcal{L}$ partitions the valid configuration space of $\mathcal{L}$ into two disjoint sets: configurations whose derived products satisfy P , and configurations whose derived products do not satisfy P .

Example 3 (Lifted SPL Analysis of SPL of Sets). Consider the domain $\mathcal{P}(\mathbb{N})$, which is the power set over the universe of the natural number $\mathbb{N}$, and the property $P(S) \equiv \exists x \in S. \text{Even}(x)$ asserting that a set $S \in \mathcal{P}(\mathbb{N})$ contains an even number. Let A be a P -analysis of $\mathcal{P}(\mathbb{N})$ and consider the SPL $\mathbf{S} = \langle F, \Phi, S, \pi \rangle$ from Example 1. Executing a lifted analysis $A^\uparrow(\mathbf{S})$ would return $\langle \Phi_\top, \Phi_\perp \rangle$ such that $\llbracket \Phi_\top \rrbracket = \{\{1, 2\}, \{2, 3\}, \{2, 4\}, \{1, 2, 3\}\}$ (highlighted in green in Fig. 1) and $\llbracket \Phi_\perp \rrbracket = \{\{1\}, \{3\}, \{1, 3\}\}$ (highlighted in red in Fig. 1).

2.2 Partial Orders, Monotonicity, and Closures

A partial order over a domain $\mathbb{D}$ is a binary reflexive, transitive and antisymmetric relation [4]. Given two partially ordered domains $\langle \mathbb{D}_1, \leq \rangle$, $\langle \mathbb{D}_2, \sqsubseteq \rangle$, a function $f : \mathbb{D}_1 \rightarrow \mathbb{D}_2$ is *monotone* iff for all $X, Y \in \mathbb{D}_1$ with $X \leq Y$, we have $f(X) \sqsubseteq f(Y)$.

Definition 3 (Monotone Property). Let $\langle \mathbb{D}, \leq \rangle$ and $\langle \{\top, \perp\}, \sqsubseteq \rangle$ be two partially ordered domains, where $\perp \sqsubseteq \top$. A property $P : \mathbb{D} \rightarrow \{\top, \perp\}$ is *monotone* with respect to $\langle \mathbb{D}, \leq \rangle$ iff for all $X, Y \in \mathbb{D}$ with $X \leq Y$, we have $P(X) \Rightarrow P(Y)$. Equivalently, for all $X, Y \in \mathbb{D}$ with $X \leq Y$, we have $\neg P(Y) \Rightarrow \neg P(X)$.

Example 4 (Monotone Properties of Sets). Consider the domain $\mathcal{P}(\mathbb{N})$ which is the power set over the universe of the natural numbers $\mathbb{N}$, and the property $\exists x \in S. \text{Even}(x)$ from Ex. 3. This property is monotone with respect to $\langle \mathcal{P}(\mathbb{N}), \subseteq \rangle$ since if S contains an even number and $S \subseteq T$, then T will contain an even number (see Fig. 1). Equivalently, if S does not contain an even number and $T \subseteq S$, then T will not contain an even number. Dually, consider the property of a set $S \in \mathcal{P}(\mathbb{N})$ containing *only* even numbers, that is, $\forall x \in S. \text{Even}(x)$. This property is monotone with respect to domain $\langle \mathcal{P}(\mathbb{N}), \supseteq \rangle$, since if S only contains even numbers and $S \supseteq T$, then T will only contain even numbers; but if S contains at least one odd number and $T \supseteq S$, then T will also contain at least one odd number.

More generally, given any universe U and property p of elements of U , the witness property $\exists x \in S. p(x)$ is monotone with respect to $\langle \mathcal{P}(U), \subseteq \rangle$, while the invariance property $\forall x \in S. p(x)$ is monotone with respect to $\langle \mathcal{P}(U), \supseteq \rangle$. Note that in Ex. 4, the same domain $\mathcal{P}(\mathbb{N})$ admits two dual orderings (subset inclusion $\subseteq$ and superset inclusion $\supseteq$), with dual monotonicity properties. This duality generalizes to other domains as follows:

Lemma 1. *Let $\langle \mathbb{D}, \leq \rangle$ be a partially ordered domain, $P : \mathbb{D} \rightarrow \{\top, \perp\}$ a property, and $\geq$ the partial order on $\mathbb{D}$ defined as $X \geq Y \Leftrightarrow Y \leq X$ for $X, Y \in \mathbb{D}$. Then P is monotone w.r.t. $\langle \mathbb{D}, \leq \rangle$ iff $\neg P$ is monotone w.r.t. $\langle \mathbb{D}, \geq \rangle$.*

For our second example, based on contracts, we can define a partial order (*contract refinement*) on the set of all AG contracts $\mathbb{K}$ as follows for contracts $K, K' \in \mathbb{K}$: $K' \leq K$ if and only if the assumptions of K are *not weaker* (i.e., not easier to satisfy) than the assumptions of K' , and the guarantees of K are *not stronger* (i.e., not harder to satisfy) than the guarantees of K' . Furthermore, a property $R : \mathbb{K} \rightarrow \{\top, \perp\}$ of AG contracts is *realizability* [10]: for a contract $K \in \mathbb{K}$, $R(K) = \top$ if there exists a system for which, in any environment satisfying its assumptions, all executions of the system will satisfy its guarantees, and $R(K) = \perp$, otherwise. Note that a realizable contract cannot be made unrealizable by strengthening assumptions or weakening guarantees, and an unrealizable contract cannot be made realizable by weakening assumptions or strengthening guarantees; thus contract realizability is monotonic with respect to $\langle \mathbb{K}, \leq \rangle$.

Example 5 (Monotonicity in AG Contracts). Consider the domain of AG contracts $\mathbb{K}$, ordered by contract refinement $\leq$, and the SPL of AG contracts $\mathbf{K}_{\text{auv}}$ for AUVs from Ex. 2. As an example, consider $\{\{A\}, \{G_1, G_2\}\} = K_{c_2} \leq K_{c_1} = \{\{A\}, \{G_2\}\}$. If we can find a system such that it satisfies the guarantees $\{G_1, G_2\}$ under the assumption A (i.e., the contract K_{c_2} is realizable), then it also satisfies the guarantee G_2 under the assumption A (i.e., the contract K_{c_1} is realizable).

Given an element X of a partially ordered domain, we refer to sets of “larger” or “smaller” elements relative to X in terms of *closures*. These are central concepts

in MONOPOLY for reasoning about the pruning of configurations during product line analysis.

Definition 4 (Upward and Downward Closure [4]). Let $\langle \mathbb{D}, \leq \rangle$ be a partially ordered domain. For any $X \in \mathbb{D}$, the *upward closure* of X with respect to $\leq$, denoted $X_{\leq}^{\uparrow}$, is the set $\{Y \in \mathbb{D} \mid X \leq Y\}$. The *downward closure* of X with respect to $\leq$, denoted $X_{\leq}^{\downarrow}$, is the set $\{Y \in \mathbb{D} \mid Y \leq X\}$.

Example 6. Consider the domain $\mathcal{P}(\mathbb{N})$ of sets of natural numbers, partially ordered by set inclusion, and fix the set $S = \{1, 2, 3, 4\}$. Its *upward closure* $S_{\subseteq}^{\uparrow}$ is the family of sets $\{S' \mid \{1, 2, 3, 4\} \subseteq S'\}$, i.e., all supersets of S . Its *downward closure* $S_{\subseteq}^{\downarrow}$ is the family of sets $\{S' \mid S' \subseteq \{1, 2, 3, 4\}\}$, i.e., all subsets of S .

Lemma 2. Let $\langle \mathbb{D}, \leq \rangle$ be a partially ordered domain, P be a monotone property with respect to $\langle \mathbb{D}, \leq \rangle$, $X \in \mathbb{D}$, and $\geq$ be the partial order on $\mathbb{D}$ defined as $X \geq Y \Leftrightarrow Y \leq X$ for $X, Y \in \mathbb{D}$. Then $X_{\leq}^{\uparrow} = X_{\geq}^{\downarrow}$ and $X_{\leq}^{\downarrow} = X_{\geq}^{\uparrow}$.

The essential relationship between monotone properties and upward/downward closures of elements in a partially ordered domain is summarized as follows:

Lemma 3. Let $\langle \mathbb{D}, \leq \rangle$ be a partially ordered domain, let P be a monotone property with respect to $\langle \mathbb{D}, \leq \rangle$, and let $X \in \mathbb{D}$. Then:

1. If $P(X)$ holds, then for all $Y \in X_{\leq}^{\uparrow}$, $P(Y)$ holds.
2. If $P(X)$ does not hold, then for all $Y \in X_{\leq}^{\downarrow}$, $P(Y)$ does not hold.

3 The MONOPOLY Framework

Intuitively, the MONOPOLY framework allows *effectively* analyzing all products of an SPL while only *directly* analyzing a subset of its products. Given some prerequisites, the result that a product X satisfies or does not satisfy a property can be generalized to all “larger” or “smaller” products, respectively. MONOPOLY is applicable whenever we have an SPL $\mathcal{L}$ over a partially ordered domain $\langle \mathbb{D}, \leq \rangle$, over which we want to analyze a property P which is monotone w.r.t. this domain, and an existing product-based P -analysis A is available. The analysis procedure proceeds iteratively: we sample a product X and use A to determine whether it satisfies P . If X satisfies P , then all “larger” products than X also satisfy P because P is monotone w.r.t. $\langle \mathbb{D}, \leq \rangle$. Thus, they do not have to be analyzed separately. Conversely, if X does not satisfy P , then all “smaller” products than X do not need to be analyzed separately because the monotonicity of P ensures that these products also do not satisfy P . Thus, by analyzing a single product X , we can get the analysis results of a set of products; namely, the products in the upward closure of X (if X satisfies P) or in the downward closure of X (if X does not satisfy P). This, of course, requires a means to determine which products are in the upward or downward closure of X .

3.1 Determining Closures of Products

A naive solution to determining the upward and downward closures of products explicitly computes and stores the partial order relations between all products ahead of time. However, this is undesirable in practice given the usually large number of products. Instead, MONOPOLY uses a *symbolic* representation (in the form of a feature expression) of the set of products contained within the upward or downward closure of a given product.

Example 7 (Closures in SPL of sets). Consider the SPL of sets $\mathbf{S}$ over the domain $\mathcal{P}(\mathbb{N})$ from Example 1 and the product $X = \{2, 3\}$, which can be derived from $\mathbf{S}$ under $\mathbf{c} = \{B, C\}$.

Suppose we want to express *symbolically*, using a feature expression, the set of products Y derivable from $\mathbf{S}$ for which we have $X \subseteq Y$ (i.e., $Y \in X_{\subseteq}^{\uparrow}$). Clearly, any such Y must be derived under a configuration satisfying $B \wedge C$ to ensure $2 \in Y$ and $3 \in Y$. Thus the feature expression $B \wedge C$ represents all supersets of X derivable from $\mathbf{S}$. Dually, if we want to represent all sets Y derivable from $\mathbf{S}$ satisfying $Y \subseteq X$, (i.e., $Y \in X_{\subseteq}^{\downarrow}$), any such Y must include neither 1 nor 4, and so must be derived under a configuration satisfying $\neg A \wedge \neg \neg(A \vee C) \equiv \neg A \wedge C$.

Ex. 7 illustrates a procedure for determining the symbolic representation of the set of products contained within the upward or downward closure of a given product. In particular, in the example we identify a relationship between (i) the partial order and (ii) the presence and absence of elements of the domain model. For example, for the fixed product $X = \{2, 3\}$ and for all products Y with $X \subseteq Y$, we should get $2 \in Y$ and $3 \in Y$. Thus, Ex. 7 suggests that, if we can identify this relationship, we can determine a procedure that uses the presence conditions of the SPL (commonly called π) to compute a feature expression representing sets of products (derivable from the SPL) that are in the upward or downward closure of a given product.

The MONOPOLY framework is developed on the conjecture that this procedure can be repeated across a variety of partially ordered SPL domains. To instantiate the MONOPOLY framework for a partially ordered domain, we need to define the operators $\text{Upward}(\cdot)$ and $\text{Downward}(\cdot)$.

Definition 5 (Upward($\cdot$) and Downward($\cdot$) Operators). Let $\langle \mathbb{D}, \leq \rangle$ be a partially ordered domain. Given any SPL $\mathcal{L} = \langle F, \Phi, D, \pi \rangle$ over $\mathbb{D}$ and a domain element $X \in \mathbb{D}$, the operator $\text{Upward}_{\leq}(X, \mathcal{L})$ produces a feature expression $\phi^{\uparrow}$ over F such that $(\mathcal{L} \downarrow \phi^{\uparrow}) \subseteq X_{\leq}^{\uparrow}$. Dually, the operator $\text{Downward}_{\leq}(X, \mathcal{L})$, returns a feature expression $\phi^{\downarrow}$ over F such that $(\mathcal{L} \downarrow \phi^{\downarrow}) \subseteq X_{\leq}^{\downarrow}$.

We emphasize a few key points with respect to these operators.

- The operators only represent the elements of the domain that are derivable from the SPL, not all elements that are in the upward or downward closure

of a given product since we are only interested in products of the SPL. For example, consider

Ex. 7. When we represent the downward closure of $\{2, 3\}$ as $\phi^\downarrow = \neg A \wedge C$, we have $(\mathcal{L} \downarrow \phi^\downarrow) = \{\{3\}, \{2, 3\}\}$. The sets $\emptyset$ and $\{2\}$ are *not* represented by $\phi^\downarrow$, despite the fact that they are both subsets of $\{2, 3\}$, because these sets cannot be derived from $\mathbf{S}$ under any feature configuration.

- The definition of the operators does not need to include evaluating partial order relations between products; at no point in Ex. 7 did we need to test set inclusion relations.
- The definition of these operators depends *only* on the nature of the domain and its partial order, not on an analysis. Once a pair of operators has been defined for a partially ordered domain, they can be reused for analysis of any property which is monotone with respect to this domain.

Finally, we note that while Def. 5 does not require it, one should ideally define $\text{Upward}(\cdot)$ and $\text{Downward}(\cdot)$ such that they compute *maximal* feature expressions. Intuitively, since the expressions $\phi^\uparrow$ and $\phi^\downarrow$ represent products whose analysis can be skipped, these feature expressions should be as general as possible; otherwise, more products may need to be analyzed directly than are strictly necessary.

Definition 6 (Maximality of $\text{Upward}(\cdot)$ and $\text{Downward}(\cdot)$). Let $\langle \mathbb{D}, \leq \rangle$ be a partially ordered domain. $\text{Upward}(\cdot)$ is *maximal* if for any SPL $\mathcal{L}$ over $\mathbb{D}$ and $X \in \mathbb{D}$ and for any feature expression ϕ such that $(\mathcal{L} \downarrow \phi) \subseteq X_\leq^\uparrow$, it holds that $\llbracket \phi \rrbracket \subseteq \llbracket \text{Upward}_\leq(X, \mathcal{L}) \rrbracket$. Dually, $\text{Downward}(\cdot)$ is *maximal* if for any SPL $\mathcal{L}$ over $\mathbb{D}$ and $X \in \mathbb{D}$, and for any feature expression ϕ such that $(\mathcal{L} \downarrow \phi) \subseteq X_\leq^\downarrow$, it holds that $\llbracket \phi \rrbracket \subseteq \llbracket \text{Downward}_\leq(X, \mathcal{L}) \rrbracket$.

Example 8 (Operators for SPLs of sets). Let $\mathcal{P}(U)$ be the power set over the universe U , partially ordered by set inclusion $\subseteq$, let $\mathbf{S} = \langle F, \Phi, S, \pi \rangle$ be an SPL over $\mathcal{P}(U)$, and let $X \in \mathcal{P}(U)$ be a domain element. Then we define

$$\text{Upward}_\subseteq(X, \mathbf{S}) = \bigwedge \{ \pi(x) : x \in X \}$$

$$\text{Downward}_\subseteq(X, \mathbf{S}) = \bigwedge \{ \neg \pi(x) : x \in S \wedge x \notin X \}$$

Example 9 (Operators for SPLs of AG contracts). Let $\mathbb{K}$ be the domain of AG contracts, partially ordered by contract refinement $\leq$, let $\mathbf{K} = \langle F, \Phi, K, \pi \rangle$ be an SPL over $\mathbb{K}$, and let $K' = \langle \{A_1, \dots, A_n\}, \{G_1, \dots, G_m\} \rangle$ be a product contract derived from $\mathbf{K}$. A contract in the *downward closure* of K' with respect to contract refinement is one whose assumptions are a *subset* of those of K' , and whose guarantees are a *superset* of those of K' . Thus, we define:

$$\text{Downward}_{\leq}(K', \mathbf{K}) = \left(\bigwedge \{ \neg \pi(A_i) \mid A_i \in \text{Asm}(K) \wedge A_i \notin \text{Asm}(K') \} \right) \\ \wedge \left(\bigwedge \{ \pi(G_i) \mid G_i \in \text{Grnt}(K') \} \right)$$

where $\text{Asm}(K)$ and $\text{Grnt}(K)$ denote the assumptions and guarantees of contract K , respectively. Dually, a contract is in the *upward closure* of K' with respect to contract refinement if its assumptions are a superset of those of K' , and its guarantees are a subset of those of K' . Thus, we define:

$$\text{Upward}_{\leq}(K', \mathbf{K}) = \left(\bigwedge \{ \pi(A_i) \mid A_i \in \text{Asm}(K') \} \right) \\ \wedge \left(\bigwedge \{ \neg \pi(G_i) \mid G_i \in \text{Grnt}(K) \wedge G_i \notin \text{Grnt}(K') \} \right)$$

Example 10 (Operators for AUV SPL). Consider the SPL of AG contracts $\mathbf{K}_{auv}$ for AUVs from Ex. 2. Then $\text{Upward}_{\leq}(K_{c_2}, \mathbf{K}_{auv}) = \pi_{auv}(A) = \text{vhf}$ and $\text{Upward}_{\leq}(K_{c_4}, \mathbf{K}_{auv}) = \text{true}$. From there, $\mathbf{K}_{auv} \downarrow \text{Upward}_{\leq}(K_{c_2}, \mathbf{K}_{auv}) = \{c_1, c_3\}$ and $\mathbf{K}_{auv} \downarrow \text{Upward}_{\leq}(K_{c_4}, \mathbf{K}_{auv}) = \{c_1, c_2, c_3, c_4\}$. Thus, if K_{c_4} is realizable, so are all other products, and if K_{c_2} is realisable, so are K_{c_1} and K_{c_3} .

3.2 The MONOPOLY Analysis Procedure

The operators $\text{Upward}(\cdot)$ and $\text{Downward}(\cdot)$ are used in MONOPOLY to lift an analysis of single products to an analysis of the SPL: Let $\langle \mathbb{D}, \leq \rangle$ be a partially ordered domain, P a monotone property w.r.t. $\langle \mathbb{D}, \leq \rangle$, and A an analysis of P . Furthermore, assume that implementations of $\text{Upward}(\cdot)$ and $\text{Downward}(\cdot)$ are given (conforming with Def. 5). Then Alg. 1 specifies a general procedure for lifting A to analyze any SPL $\mathcal{L} = \langle F, \Phi, D, \pi \rangle$ over $\mathbb{D}$. Per Def. 2, the outcome of the analysis is a partition of the configuration space $\llbracket \Phi \rrbracket$ into two disjoint sets $\llbracket \Phi_{\top} \rrbracket$ and $\llbracket \Phi_{\perp} \rrbracket$, such that all configurations satisfying $\Phi_{\top}$ correspond to products satisfying P , and those satisfying $\Phi_{\perp}$ correspond to products not satisfying P .

In Alg. 1, the feature expressions $\Phi_{\top}$ and $\Phi_{\perp}$ are constructed iteratively such that they always represent the set of configurations whose associated products have been (directly or indirectly) analyzed until this point. Both $\Phi_{\top}$ and $\Phi_{\perp}$ are initialized with $\perp$, indicating that no products have yet been analyzed (Line 1). We then incrementally analyze products until no more valid products (per the feature model Φ) are left to be analyzed (i.e., satisfy neither $\Phi_{\top}$ nor $\Phi_{\perp}$). We determine this by checking whether the propositional formula $\Phi \wedge \neg(\Phi_{\top} \vee \Phi_{\perp})$ is satisfiable (Line 2). If so, we sample a satisfying feature configuration $\mathbf{c}$ and derive the product $X = \mathcal{L}|_{\mathbf{c}}$ (Line 3). We then execute the product-based analysis A on product X and consider the result. If the analysis succeeds ($A(X) = \top$), we compute the feature expression $\phi^{\uparrow}$ representing products in the upward closure of X (Line 5) and add it to $\Phi_{\top}$ as a disjunct; this reflects that all configurations in $\llbracket \phi^{\uparrow} \rrbracket$ correspond to products which satisfy P and no longer need to be analyzed

Algorithm 1 The MONOPOLY Analysis Procedure

Require: P is monotone over $\langle \mathbb{D}, \leq, \rangle$, $A : \mathbb{D} \rightarrow \{\top, \perp\}$ is a P -analysis, and $\mathcal{L} = \langle F, \Phi, D, \pi \rangle$ is an SPL over $\mathbb{D}$

- 1: $\Phi_{\top} \leftarrow \perp, \Phi_{\perp} \leftarrow \perp$
- 2: **while** $\Phi \wedge \neg(\Phi_{\top} \vee \Phi_{\perp})$ is satisfiable **do**
- 3: $X \leftarrow \mathcal{L}|_{\mathbf{c}}$ for some $\mathbf{c} \models \Phi \wedge \neg(\Phi_{\top} \vee \Phi_{\perp})$
- 4: **if** $A(X) = \top$ **then**
- 5: $\phi^{\uparrow} \leftarrow \text{Upward}_{\leq}(X, \mathcal{L})$
- 6: $\Phi_{\top} \leftarrow \Phi_{\top} \vee \phi^{\uparrow}$
- 7: **else**
- 8: $\phi^{\downarrow} \leftarrow \text{Downward}_{\leq}(X, \mathcal{L})$
- 9: $\Phi_{\perp} \leftarrow \Phi_{\perp} \vee \phi^{\downarrow}$
- 10: **end if**
- 11: **end while**
- 12: $\Phi_{\top} \leftarrow \Phi_{\top} \wedge \Phi$
- 13: $\Phi_{\perp} \leftarrow \Phi_{\perp} \wedge \Phi$

(Line 6). If the analysis of X fails (i.e., $A(X) = \perp$), then we compute the feature expression representing products in the downward closure of X (Line 8) and perform the analogous extension of $\Phi_{\perp}$ (Line 9). Finally, upon termination of the loop, we conjoin both $\Phi_{\top}$ and $\Phi_{\perp}$ with the feature model Φ , since we only care about the analysis results for valid products.

Theorem 1. *Given a partially ordered domain $\langle \mathbb{D}, \leq \rangle$, a monotone property P with respect to $\langle \mathbb{D}, \leq \rangle$, an SPL $\mathcal{L} = \langle F, \Phi, D, \pi \rangle$ over $\mathbb{D}$, and a P -analysis A , Alg. 1 correctly lifts A per Def. 2.*

Proof. We show that the algorithm respects the invariants: (a) $(\mathcal{L} \downarrow \Phi_{\top}) \subseteq \{X \in \mathbb{D} \mid P(X)\}$ and (b) $(\mathcal{L} \downarrow \Phi_{\perp}) \subseteq \{X \in \mathbb{D} \mid \neg P(X)\}$.

These invariants hold initially since $\llbracket \perp \rrbracket = (\mathcal{L} \downarrow \perp) = \emptyset$. Suppose that after iteration i , the invariants hold, and consider iteration $i + 1$. We sample product X obtained by deriving $\mathcal{L}|_{\mathbf{c}}$ under some $\mathbf{c} \models \Phi \wedge \neg(\Phi_{\top} \vee \Phi_{\perp})$. Without loss of generality, suppose $A(X) = \top$; we then know $P(X)$ holds since A is a P -analysis. We then compute $\phi^{\uparrow} = \text{Upward}_{\leq}(X, \mathcal{L})$ such that $(\mathcal{L} \downarrow \phi^{\uparrow}) \subseteq X_{\leq}^{\uparrow}$, per Def. 5. By Lemma 3, we have $X_{\leq}^{\uparrow} \subseteq \{Y \in \mathbb{D} \mid P(Y)\}$ and therefore $(\mathcal{L} \downarrow \phi^{\uparrow}) \subseteq \{Y \in \mathbb{D} \mid P(Y)\}$. By the induction hypothesis, it holds that $(\mathcal{L} \downarrow \Phi_{\top}) \subseteq \{Y \in \mathbb{D} \mid P(Y)\}$. Therefore, we have $(\mathcal{L} \downarrow (\Phi_{\top} \vee \phi^{\uparrow})) \subseteq \{Y \in \mathbb{D} \mid P(Y)\}$, and so Line 6 maintains invariant (a). The analogous argument can be employed for the “else” case, in which $A(X) = \perp$, to show that invariant (b) is maintained.

Upon termination of the loop, we have that $\Phi \wedge \neg(\Phi_{\top} \vee \Phi_{\perp})$ is unsatisfiable, and therefore, $\llbracket \Phi \rrbracket \subseteq \llbracket \Phi_{\top} \rrbracket \cup \llbracket \Phi_{\perp} \rrbracket$. Conjoining $\Phi_{\top}$ and $\Phi_{\perp}$ with Φ (Lines. 12-13), ensures that the procedure finishes with $\llbracket \Phi_{\top} \rrbracket \cup \llbracket \Phi_{\perp} \rrbracket = \llbracket \Phi \rrbracket$. Invariants (a) and (b) imply that $\Phi_{\top}$ and $\Phi_{\perp}$ represent disjoint sets of configurations. Finally, the requirement that $\Phi_{\top}$ and $\Phi_{\perp}$ represent sound analysis results is provided by invariants (a) and (b). $\square$

Example 11 (Alg. 1 for SPL of Sets). Consider the domain $\mathcal{P}(\mathbb{N})$ over the universe $\mathbb{N}$ with partial order $\subseteq$ and the property $P(S) \equiv \exists x \in S. \text{Even}(x)$, asserting that $S \in \mathcal{P}(\mathbb{N})$ contains an even number. Then P is monotone with respect to $\langle \mathcal{P}(\mathbb{N}), \subseteq \rangle$ (see Ex. 4). Let A be a P -analysis. We illustrate the idea of Alg. 1 using the SPL $\mathbf{S} = \langle F, \Phi, S, \pi \rangle$ over $\mathcal{P}(\mathbb{N})$ from Ex. 1. We start by sampling a product, e.g., $X = \{1, 3\}$. Then $A(X) = \perp$. Using Ex. 8, we update $\Phi_{\perp}$ with $\neg B \wedge (A \vee C)$. This means that we do not have to analyze the products $\{1\}$ and $\{3\}$ anymore. Next, we choose the product $X = \{1, 2\}$. Then $A(X) = \top$. Again using Ex. 8, we update $\Phi_{\top}$ with $A \wedge B$. This implies that we do not have to analyze the product $\{1, 2, 3\}$ anymore. Next, we do the same with the products $\{2, 3\}$ and $\{2, 4\}$, adding $B \wedge C$ and $B \wedge \neg(A \vee C)$ to $\Phi_{\top}$, before the loop terminates. This gives $\Phi_{\top} = (A \wedge B) \vee (B \wedge C) \vee (B \wedge \neg(A \vee C))$ and $\Phi_{\perp} = \neg B \wedge (A \vee C)$ as already seen in Ex. 3.

Using domain knowledge, we chose the configurations in Ex. 11 such that we have to do as few analyses of products as possible. However, our implementation of MONOPOLY does not yet use this kind of domain knowledge for choosing configurations. Instead, configurations are chosen by the satisfiability solver, leaving room for improvement by defining suitable heuristics (see also Sec. 4.4).

However, there is a theoretically optimal sampling strategy for MONOPOLY: Let $\mathcal{L}$ be an SPL with feature model Φ over a partially ordered domain, and P a monotone property of elements of the domain. The product space of $\mathcal{L}$ can be partitioned into an *upper* set U of products satisfying P , and a *lower* set L of products not satisfying P . There exist subsets $U' \subseteq U$ and $L' \subseteq L$ such that U' and L' are the smallest sets of products such that, for every product $X \in U$, there is a product $Y \in U'$ such that $X \in Y_{\leq}^{\uparrow}$, and for every product $X \in L$, there is a product $Y \in L'$ such that $X \in Y_{\leq}^{\downarrow}$. Since L' and U' are minimal, an optimal sampling strategy for MONOPOLY would sample exactly those products in $L' \cup U'$. Moreover, by their minimality, any run of MONOPOLY must analyze at least $|L'| + |U'|$ products to cover the entire product space.

3.3 Implementation

We provide an implementation⁵ of the MONOPOLY framework as a Python package, which includes a polymorphic implementation of Alg. 1. In our implementation, annotative SPL domains (variational sets, contracts, etc.) are defined

⁵ <https://github.com/monopoly-spl/monopoly>

as extensions of an abstract class `Variational`. A concrete subclass of `Variational` must provide an implementation of the product derivation operation (i.e., how a product can be obtained from the SPL) as well as the `Upward(·)` and `Downward(·)` operators for the desired annotative SPL domain. Once these have been defined, the implementation of Alg. 1 can be executed to lift any monotone Boolean analysis over any SPL of that domain. The analysis itself must be provided as a Boolean-valued Python function, but this can be simply a wrapper for an external analysis tool, if needed. Our implementation also provides support for measuring the expressiveness of the feature expressions computed for upward and downward closures by counting the number of valid configurations satisfying a given feature expression. Propositional reasoning is performed using the Z3 theorem prover [5].

4 Empirical Analysis

In our empirical analysis, we apply MONOPOLY to SPLs from existing benchmarks. We are primarily interested in evaluating the usefulness of pruning configurations by approximating upward and downward closures using feature expressions. On one hand, in a realistic setting where an entire SPL should be analyzed, we want to measure how many products need to be analyzed directly and how many analysis results can be deduced (i.e., how many configurations can be pruned). On the other hand, given a product analysis, we are interested in how many configurations can be pruned from this specific result. In particular, we examine how this pruning of configurations varies across the products of an SPL. This leads to the following research questions:

- **RQ1:** Using MONOPOLY and given an SPL, how many analyses have to be performed to deduce analysis results for every product of the SPL?
- **RQ2:** Using MONOPOLY and given an analysis of a product of an SPL, how many analysis results can be deduced?

4.1 Subject Domain: SPLs of Metamodels

In this evaluation, we consider the domain of *modeling languages* (i.e., metamodels). In this context, a metamodel is used to describe a language of models, such as class diagrams or state machines. A metamodel defines a set of *classes*, each of which contains a number of *fields*, such as attributes and references to other classes, and which may inherit from other classes in the metamodel.

Variability can naturally emerge in a metamodeling context when new language constructs are introduced to model novel phenomena, or when some aspects of a language are not needed to solve a given problem. The notion of defining *SPLs of metamodels* has accordingly been studied [13, 7], with various techniques for modeling and analyzing these SPLs. The metamodeling domain provides a natural testbed for the MONOPOLY framework, since metamodels admit a partial ordering which can be reduced to structural criteria. In particular, we consider

the *subsumption relation* $M_1 \sqsubseteq M_2$ between metamodels M_1 and M_2 , which holds when every instance (model) of M_1 is also an instance (model) of M_2 . This relation can be reduced to checking (i) that all classes of M_1 are also classes of M_2 ; (ii) that all fields of these classes in M_1 have matching definitions in M_2 ; and (iii) all inheritance relations in M_1 are present in M_2 . We follow the annotative metamodel representation defined by Guerra et al. [13] in which classes, their fields, and inheritance links are annotated by presence conditions, indicating the set of language variants in which these elements are present.

4.2 Methodology

SPL Subjects. We evaluate the MONOPOLY framework on two SPL case studies from the benchmark developed by Guerra et al. [13].

- The Automata SPL defines a family of languages for representing different kinds of automata (probabilistic automata, automata over infinite strings, etc.). The Automata SPL includes six classes, with 18 presence conditions expressed over an alphabet of 20 features, and with a feature model admitting 2016 valid configurations.
- The GPL SPL represents a family of languages for defining graphs and supporting various graph operations (e.g., search procedures or computing minimum spanning trees). The GPL SPL includes 16 classes, with 42 presence conditions expressed over an alphabet of 38 features, and with a feature model admitting 840 valid configurations.

To analyze these SPLs with our implementation of MONOPOLY, we created a Python representation of MOF-style metamodels and manually translated the annotated metamodels from [13] to this format. To instantiate MONOPOLY for this domain, we implemented the required `Upward(·)` and `Downward(·)` operators analogously to their implementations for the SPL of sets (Ex. 8). That is, to compute the upward closure of a metamodel product, we conjoin the presence conditions of its classes, their fields, and its inheritance relations; to compute the downward closure, we conjoin the negations of the presence conditions of classes, fields, and inheritance relations which do *not* exist in the product.

Target Analysis. We consider a monotone analysis A over metamodels which determines whether the metamodel contains at least k classes, each of which containing at least m fields. This is a simple example of a monotone analysis which may nonetheless be useful, for instance, to determine the complexity of the language and identify potential areas for modularization. We set $k = 2$ and $m = 4$ in analyzing both SPLs. Finally, note that in our initial implementation of MONOPOLY, product sampling (Line 3 of Alg. 1) is performed by simply extracting a satisfying configuration from Z3, without any additional sampling heuristics or strategies.

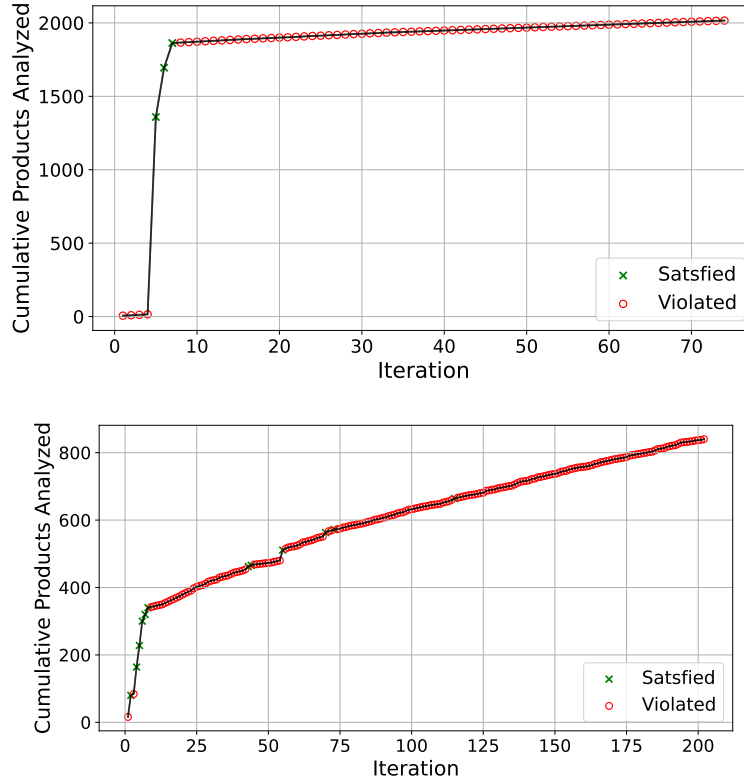


Fig. 2: Cumulative number of configurations analyzed after n iterations of the analysis loop for the Automata SPL (top) and the GPL SPL (bottom).

4.3 Results

With respect to **RQ1**, we are able to analyze the entire configuration space of the Automata SPL after directly analyzing 74 out of the 2016 products (**3.6%**). Fig. 2 (top) illustrates the cumulative number of configurations that are analyzed (directly or indirectly) after each analysis invocation; crosses denote a positive analysis ($\top$) and circles a negative analysis ($\perp$). Three analysis results (all on products satisfying the property) are able to significantly increase the cumulative configuration coverage, to the effect that over 90% of the configuration space is covered after analyzing only 14 products. After this point, we see a much more gradual process of analyzing the remaining configurations.

For the GPL SPL, we are able to analyze the entire configuration space after directly analyzing 202 out of 840 products (**24%**). The cumulative configuration coverage is shown in Fig. 2 (bottom). In this SPL, we again see a cluster of analyses early on which provide several jumps in coverage, albeit at a more moderate

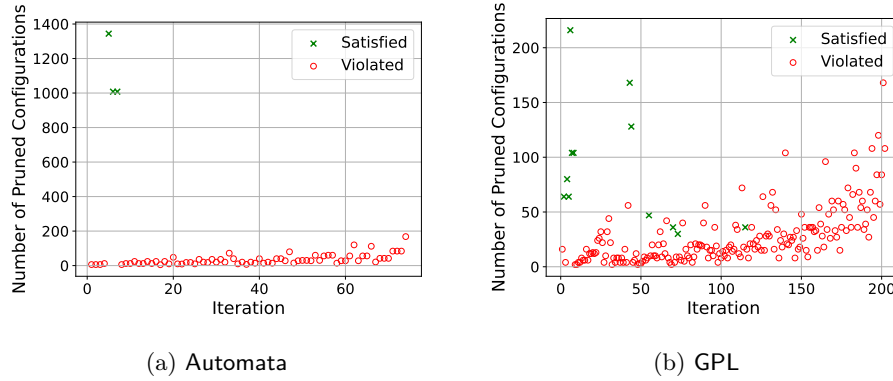


Fig. 3: Total number of pruned configurations for each analysis loop iteration.

increase than in the Automata SPL. After the first eight analysis invocations, there is again a more gradual ascent to complete coverage, with each analysis invocation still generalizing to a handful of new configurations. Thus, we observe a high variance in the number of analysis invocations needed to achieve 100% configuration coverage, ranging from 3.6% of the total configuration space for the Automata SPL to 24% for the GPL SPL.

While Fig. 2 illustrates cumulative coverage, to investigate **RQ2**, we report the total number of valid configurations represented by the feature expressions computed by the `Upward(.)` and `Downward(.)` operators following each analysis invocation. Note that we only measure the number of pruned configurations resulting from those products we happen to sample during the MONOPOLY procedure. The results for Automata and GPL SPLs are shown in Fig. 3. In both cases, there is a relatively small number of analysis results that enable pruning a large number of configurations, even though the extent varies between the SPLs. In the Automata SPL (Fig. 3a), there are three products analyzed that enable pruning between 44% and 68% of the total configuration space; in the GPL SPL (Fig. 3b), there is a larger set of products which enables pruning between 10% and 27% of the total configuration space.

To summarize: with respect to **RQ1**, our experiments indicate that the MONOPOLY framework can eliminate a significant number (i.e. a majority) of products to be analyzed relative to a naive strategy, although the precise degree of speedup can vary significantly between product line models (e.g., 3.6%–24%). With respect to **RQ2**, our experiments indicate that the amount of configurations that be pruned via MONOPOLY following the direct analysis of a product varies significantly across the product space; some analysis results may not generalize to any additional products, while others generalize to even a majority of the total product space. Nevertheless, we again observe significant variance between our subject metamodels, with the Automata metamodel having a very small number

of products with significant pruning capacity, and the GPL metamodel providing a larger number of products with more moderate pruning capacity.

4.4 Discussion

As with any form of variability-aware analysis, the MONOPOLY framework is highly sensitive to the degree of similarity between products. More specifically, the efficacy of MONOPOLY is constrained by the shape of the partial order between products. In a hypothetical worst-case, in which no two products are partially ordered, the MONOPOLY framework devolves into a naive product-based analysis. In our empirical analysis, however, we have demonstrated that there exist nontrivial SPLs for which the partial ordering between products enables – in some cases extensive – generalization of analysis results between products. At the same time, the variance in the degree of speedup between the Automata and GPL SPLs reflects the sensitivity of MONOPOLY to the shape of the ordering. We note that our experiments are limited to only two SPLs from the metamodeling domain which, while nontrivial, are academic examples that may not necessarily reflect the characteristics of industrial SPLs. Therefore, further experiments with a more diverse range of SPL domains, SPL models, and analyses are necessary to determine the general applicability of MONOPOLY.

Another factor which can significantly impact the efficacy of MONOPOLY is the choice of products which are sampled for analysis, and in which order they are sampled. As mentioned above, our preliminary experiments with MONOPOLY did not use any sophisticated sampling strategy. In our empirical analysis, we can clearly observe that this strategy is non-optimal. This is most evident in the analysis of the GPL SPL (Fig. 3b) where the analysis of the last few products enables pruning more configurations than the analysis of products analyzed earlier. This represents an obvious inefficiency in the implementation; had these “more useful” products been analyzed earlier, the entire run would have terminated sooner, with fewer products needing to be analyzed. While we did not study the effectiveness of various sampling strategies in this work, we believe this presents a clear and measurable research problem for follow-on work.

5 Related Work

There are numerous strategies which can be undertaken to make the analysis of SPLs tractable. Thüm et al. [27] provide a classification of these strategies based on the level of abstraction at which analysis is done. Per this classification, MONOPOLY represents an *optimized product-based* analysis strategy, as we aim to analyze products directly using traditional methods, and identify opportunities to reuse these analysis results for other products. Other optimized, product-based analysis strategies have been proposed for particular SPL domains. In the context of program verification, Bruns et al. employ a slicing procedure for identifying opportunities to reuse proofs of correctness between products [2]. Proof reuse between products has also been studied via *partial proofs* [16], in

which an incomplete proof which can be shared across all products is created first, and subsequently completed for each product. In the context of software model checking, methods of reusing verification results across products have also been studied. Cordy et al. [3] provide a method for checking whether the addition of a feature to a product transition system preserves a previous verification result. In the context of controller synthesis, Greenyer et al. have developed an incremental synthesis strategy in which reuse between “similar” products is determined based on the feature diagram representation of the feature model [12].

More generally, a strategy for analyzing an SPL reuses product-based analyses are referred to as *black-box lifts*, following the terminology of [19]. One can further distinguish between black-box lifts which perform a single analysis iteration (typically on a *metaproduct* or *product simulator*), and those which perform incremental analysis, such as MONOPOLY. In addition to the examples given above, incremental black-box lifting has been developed for testing [15], model transformations [24], program equivalence checking [29], theorem proving [30], and other analysis problems – we refer to [19] for a more comprehensive survey. We emphasize that MONOPOLY is a *lifting framework* that can be applied to any analysis over a partially ordered product line domain, rather than focusing on a particular form of analysis.

Finally, while our initial implementation of MONOPOLY relies on a theorem prover to sample configurations, without any further sampling heuristic, there is extensive literature on product sampling for SPLs – we refer to [28] for a survey. Investigating improvements to MONOPOLY via thoughtful product sampling strategies is a promising direction for future work.

6 Conclusion

In this paper, we presented MONOPOLY, a generic framework for the analysis of SPLs. MONOPOLY is based on the intuition that the analysis result of a single product often generalizes to sets of products. MONOPOLY aims to operationalize this intuition by exploiting monotonicity of properties to prune configurations, generalizing analysis results of single products to sets of products. This is aided through the symbolic representation of configurations as feature expressions. With MONOPOLY, existing analysis techniques can be re-used out-of-the-box, in a black-box fashion. Instantiating MONOPOLY in a given SPL domain can be done by providing a method for approximating upward and downward closures as feature expressions. We have shown how to instantiate MONOPOLY for three SPL domains (sets, contracts, and metamodels), and demonstrated the feasibility of MONOPOLY and its ability to reduce the analysis workload for nontrivial SPLs. We propose as future work an investigation into improvements of MONOPOLY via product sampling strategies.

Acknowledgments. This research was partially funded by General Motors and NSERC Alliance, the WWTF ICT22-007 grant ForSmart and the SBA Research COMET Center SBA-K1 NGC managed by the FFG.

References

1. Benavides, D., Sundermann, C., Feichtinger, K., Galindo, J.A., Rabiser, R., Thüm, T.: Uvl: Feature modelling with the universal variability language. *Journal of systems and software* **225**, 112326 (2025)
2. Bruns, D., Klebanov, V., Schaefer, I.: Verification of software product lines with delta-oriented slicing. In: *International Conference on Formal Verification of Object-Oriented Software*. pp. 61–75. Springer (2010)
3. Cordy, M., Schobbens, P.Y., Heymans, P., Legay, A.: Towards an incremental automata-based approach for software product-line model checking. In: *Proceedings of the 16th International Software Product Line Conference-Volume 2*. pp. 74–81 (2012)
4. Davey, B.A., Priestley, H.A.: *Introduction to lattices and order*. Cambridge university press (2002)
5. De Moura, L., Bjørner, N.: Z3: An efficient smt solver. In: *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. pp. 337–340. Springer (2008)
6. Delaware, B., Cook, W., Batory, D.: Product lines of theorems. In: *Proceedings of the 2011 ACM international conference on Object oriented programming systems languages and applications*. pp. 595–608 (2011)
7. Di Sandro, A., Shahin, R., Chechik, M.: Adding product-line capabilities to your favourite modeling language. In: *Proceedings of the 17th International Working Conference on Variability Modelling of Software-Intensive Systems*. pp. 3–12 (2023)
8. Eriksson, M., Börstler, J., Borg, K.: Managing requirements specifications for product lines—an approach and industry case study. *Journal of Systems and Software* **82**(3), 435–447 (2009)
9. Erwig, M., Walkingshaw, E., Chen, S.: An abstract representation of variational graphs. In: *Proceedings of the 5th International Workshop on Feature-Oriented Software Development*. pp. 25–32 (2013)
10. Gacek, A., Katis, A., Whalen, M.W., Backes, J., Cofer, D.: Towards realizability checking of contracts using theories. In: *NASA Formal Methods Symposium*. pp. 173–187. Springer (2015)
11. Garmendia, A., Guerra, E., de Lara, J.: Product lines of graphical modelling languages. In: *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*. pp. 69–79 (2024)
12. Greenyer, J., Brenner, C., Cordy, M., Heymans, P., Gressi, E.: Incrementally synthesizing controllers from scenario-based product line specifications. In: *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*. pp. 433–443 (2013)
13. Guerra, E., de Lara, J., Chechik, M., Salay, R.: Property satisfiability analysis for product lines of modelling languages. *IEEE Transactions on Software Engineering* **48**(2), 397–416 (2020)
14. Kenner, A., Kästner, C., Haase, S., Leich, T.: Typechef: Toward type checking `#ifdef` variability in c. In: *Proceedings of the 2nd international workshop on feature-oriented software development*. pp. 25–32 (2010)
15. Kim, C.H.P., Marinov, D., Khurshid, S., Batory, D., Souto, S., Barros, P., d’Amorim, M.: Splat: Lightweight dynamic analysis for reducing combinatorics in testing configurable systems. In: *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*. pp. 257–267 (2013)
16. Kodetzki, M., Bordis, T., Runge, T., Schaefer, I.: Partial proofs to optimize deductive verification of feature-oriented software product lines. In: *Proceedings of*

- the 18th International Working Conference on Variability Modelling of Software-Intensive Systems. pp. 17–26 (2024)
17. Lee, K., Kang, K.C., Lee, J.: Concepts and guidelines of feature modeling for product line software engineering. In: International Conference on Software Reuse. pp. 62–77. Springer (2002)
 18. Murphy, L., Carwehl, M., Päckler, J., Chechik, M.: Towards systematic maintenance of assurance for evolving self-adaptive systems. In: 2025 IEEE Engineering Reliable Autonomous Systems (ERAS). pp. 1–8. IEEE (2025)
 19. Murphy, L., Saifi, M., Di Sandro, A., Chechik, M.: A structural taxonomy for lifted software product line analyses. *Journal of Systems and Software* **222**, 112280 (2025)
 20. Murphy, L., Viger, T., Di Sandro, A., Babikian, A., Chechik, M.: Assurance case development for evolving software product lines: A formal approach. *Formal Aspects of Computing* (2026)
 21. Nešić, D., Nyberg, M., Gallina, B.: Product-line assurance cases from contract-based design. *Journal of Systems and Software* **176**, 110922 (2021)
 22. Pohl, K., Böckle, G., van der Linden, F.J.: *Software Product Line Engineering: Foundations, Principles, and Techniques*. Springer, Germany (2005). <https://doi.org/10.1007/3-540-28901-1>
 23. Post, H., Sinz, C.: Configuration lifting: Verification meets software configuration. In: 2008 23rd IEEE/ACM International Conference on Automated Software Engineering. pp. 347–350. IEEE (2008)
 24. Salay, R., Famelis, M., Rubin, J., Di Sandro, A., Chechik, M.: Lifting model transformations to product lines. In: Proceedings of the 36th International Conference on Software Engineering. pp. 117–128 (2014)
 25. Sangiovanni-Vincentelli, A., Damm, W., Passerone, R.: Taming dr. frankenstein: Contract-based design for cyber-physical systems. *European journal of control* **18**(3), 217–238 (2012)
 26. Shahin, R., Chechik, M.: Automatic and efficient variability-aware lifting of functional programs. *Proceedings of the ACM on Programming Languages* **4**(OOPSLA), 1–27 (2020)
 27. Thüm, T., Apel, S., Kästner, C., Schaefer, I., Saake, G.: A classification and survey of analysis strategies for software product lines. *ACM Computing Surveys (CSUR)* **47**(1), 1–45 (2014)
 28. Varshosaz, M., Al-Hajjaji, M., Thüm, T., Runge, T., Mousavi, M.R., Schaefer, I.: A classification of product sampling for software product lines. In: Proceedings of the 22nd International Systems and Software Product Line Conference-Volume 1. pp. 1–13 (2018)
 29. Wang, A., Feng, N., Chechik, M.: Code-level functional equivalence checking of annotative software product lines. In: Proceedings of the 27th ACM International Systems and Software Product Line Conference-Volume A. pp. 64–75 (2023)
 30. Young, J.M., Bittner, P.M., Walkingshaw, E., Thüm, T.: Variational satisfiability solving: efficiently solving lots of related sat problems. *Empirical Software Engineering* **28**(1), 14 (2023)